

Laboratorium nr 1 – parser strony www

Pozyskanie danych jest jednym z podstawowych wyzwań w analizie danych. Gdy dane nie są dostępne przez API, buduje się własny parser stron internetowych. W ramach tego laboratorium przygotujemy taki parser — ale zamiast obciążać cudze serwery, każda para zbuduje sobie nawzajem serwis do sparsowania.

Taka formuła ma dwie zalety. Nie narusza niczyjego regulaminu ani infrastruktury, a przy tym uczy obu stron zagadnienia: żeby zbudować stronę, którą da się sparsować, trzeba rozumieć strukturę dokumentu; żeby ją sparsować, trzeba rozumieć selektory.

Umiejętności z tego laboratorium będziecie w przyszłości stosować do prawdziwych serwisów. Pamiętajcie wtedy o trzech rzeczach: dane mogą być chronione prawem autorskim, ochroną baz danych lub regulaminem serwisu; plik robots.txt i regulamin trzeba przeczytać przed pierwszym żądaniem; a w kodzie zawsze kodujemy opóźnienie między żądaniami — inaczej zostaniemy zablokowani, i słusznie.

Organizacja pracy

Pracujecie w parach. Każda osoba występuje w jednej z ról:

- jako AUTOR — przygotowuje serwis, który sparsuje partner;
- jako PARSER — pisze scraper do serwisu przygotowanego przez partnera.

Poziom trudności (3.0 / 4.0 / 5.0) para wybiera wspólnie.

Serwis wolno (i warto) wygenerować z wykorzystaniem LLM — dane produktów mogą być fikcyjne lub losowe;

Poziomy 4.0 i 5.0 zawierają listy wyboru: realizujecie wskazaną liczbę pozycji, a nie całą listę. Każda lista kończy się punktem otwartym — jeśli macie własny pomysł o zbliżonym poziomie trudności śmiało go zrealizujcie.

Zadanie na 3.0

Rola AUTOR

Przygotuj jedną stronę HTML zawierającą katalog produktów.

- minimum 60 rekordów;
- minimum 10 atrybutów każdego rekordu, na przykład: nazwa, cena, kategoria, producent, dostępność, ocena, liczba opinii, data dodania, gwarancja, odnośnik do produktu;
- rekordy zapisane w dwóch różnych strukturach — część jako tabela, część jako powtarzalne karty (article albo div);
- nazwy klas i identyfikatorów semantyczne (cena, produkt, data-id) — bez nazw generowanych typu a3jX9;
- strona statyczna, bez JavaScriptu.

Rola PARSER

Napisz parser serwisu partnera, który zapisze dane do pliku CSV lub XLSX.

- jeden wiersz = jeden rekord, jedna kolumna = jeden atrybut;
- minimum 60 wierszy — z obu struktur, sprowadzonych do wspólnego formatu;
- cena zapisana jako liczba, nie jako tekst z symbolem waluty;
- kodowanie UTF-8, poprawne polskie znaki;
- opóźnienie między żądaniami.

Poziom 3.0 nie wymaga sprawozdania ani odpowiedzi ustnej.

Zadanie na 4.0

Rola AUTOR

Wymagania obowiązkowe

- wszystko z poziomu 3.0;
- paginacja — minimum 5 podstron, łącznie minimum 200 rekordów;
- podstrony szczegółów — każdy rekord ma własną stronę z minimum 8 dodatkowymi atrybutami (łącznie minimum 18 atrybutów).

Wybierz co najmniej 3 utrudnienia

- ceny zapisane jako „1 299,00 zł”, a u części rekordów w innej walucie;
- braki wartości w dwóch różnych polach, u co najmniej 15% rekordów;
- minimum 8 zduplikowanych rekordów rozrzuconych po różnych podstronach;
- tabela zapisana bez znacznika tbody oraz kilkanaście niezamkniętych znaczników li lub p;
- niespójne formaty dat w tym samym polu (2026-08-26, 26.08.2026, „wczoraj”);
- część atrybutów dostępna wyłącznie w atrybutach data-*, a nie w treści elementu;
- encje i znaki specjalne w treści (, ´, twarde spacje w liczbach);
- dwie podstrony o odmiennej strukturze niż pozostałe — parser musi wykryć obie;
- własne utrudnienie o zbliżonym poziomie trudności.

Rola PARSER

Wymagania obowiązkowe

- minimum 200 rekordów i minimum 18 atrybutów;
- parser przechodzi przez wszystkie podstrony i wchodzi w podstrony szczegółów, łącząc dane z listy i ze szczegółów;
- czyszczenie danych: ceny zamienione na liczby, ujednolicone formaty, usunięte duplikaty;
- brak danych obsługiwany jako pusta wartość, a nie jako wyjątek przerywający działanie;
- opóźnienie minimum 0,5 sekundy między zadaniami.

Wybierz co najmniej 2 rozszerzenia

- walidacja typów wszystkich kolumn i raport z liczbą odrzuconych wartości;
- eksport do dwóch formatów jednocześnie (np. CSV oraz XLSX lub JSON);
- logowanie przebiegu do pliku wraz z podsumowaniem: liczba stron, rekordów, błędów, czas działania;
- minimum 3 testy jednostkowe funkcji czyszczących dane;
- pobieranie równoległe z ograniczeniem liczby jednoczesnych żądań;
- parametryzacja z linii poleceń (adres serwisu, liczba stron, plik wyjściowy);
- własne rozszerzenie o zbliżonym poziomie trudności.

Ponadto: analiza danych i sprawozdanie zgodnie z rozdziałem „Analiza danych” oraz odpowiedź ustna.

Zadanie na 5.0

Rola AUTOR

Wymagania obowiązkowe

- wszystko z poziomu 4.0;

- treść wstrzykiwana JavaScriptem — rekordy trzymane w pliku JSON i dodawane do dokumentu przez fetch; w źródle HTML (Ctrl+U) rekordów ma NIE być;
- doładowywanie porcjami — przycisk „Wczytaj więcej” lub doładowanie przy przewijaniu;
- minimum 400 rekordów i minimum 22 atrybuty;
- sztuczne opóźnienie 300–2000 ms przy doładowywaniu porcji.

Wybierz co najmniej 2 utrudnienia

- minimum jeden atrybut widoczny dopiero po interakcji — kliknięcie zakładki, rozwinięcie details;
- filtrowanie lub sortowanie zmieniające zawartość bez przeładowania strony;
- prosty serwer (Python lub Node) zamiast http.server, zwracający kod 429 po przekroczeniu limitu żądań, wraz z nagłówkiem Retry-After;
- część rekordów renderowana dopiero po przewinięciu do danego miejsca strony (leniwe ładowanie);
- losowa kolejność atrybutów w karcie produktu — parser nie może polegać na pozycji;
- własne utrudnienie o zbliżonym poziomie trudności.

Rola PARSER

Wymagania obowiązkowe

- parser zbudowany w oparciu o Selenium (bądź dowolne inne narzędzie);
- jawne oczekiwanie na elementy — WebDriverWait z warunkiem, nie sleep na sztywno;
- obsługa doładowywania aż do wyczerpania rekordów;
- ponawianie nieudanych operacji z opóźnieniem wykładniczym (1 s, 2 s, 4 s...) i górnym limitem prób;
- w sprawozdaniu: porównanie liczby rekordów widocznych w surowym HTML (pobranym bez przeglądarki) i w drzewie DOM po wykonaniu skryptów — z konkretnymi liczbami i wnioskiem.

Wybierz co najmniej 2 rozszerzenia

- obsługa kodu 429 z poszanowaniem nagłówka Retry-After;
- automatyczny zrzut ekranu i zapis źródła strony przy każdym błędzie, do celów diagnostycznych;
- tryb headless wraz z pomiarem czasu działania w obu trybach;
- wznowianie przerwanej sesji od ostatniego zapisanego punktu (checkpoint);
- porównanie czasu działania i zużycia pamięci: podejście bez przeglądarki kontra Selenium, na tym samym podzbiore danych;
- wykrycie i wykorzystanie źródła danych, z którego korzysta strona (plik JSON), wraz z omówieniem, dlaczego jest to szybsze;
- własne rozszerzenie o zbliżonym poziomie trudności, uzgodnione z prowadzącym.

Analiza danych (poziomy 4.0 i 5.0)

Kontrola jakości danych

Tę część wykonaj jako pierwszą — służy sprawdzeniu, czy parser zadziałał poprawnie.

- liczba pozyskanych rekordów i liczba braków w każdej kolumnie (w sztukach i procentach);
- liczba duplikatów przed odrzuceniem i po odrzuceniu;
- typy kolumn — sprawdź, czy kolumny liczbowe faktycznie są liczbowe;
- wartości odstające wraz z interpretacją: czy to prawdziwa anomalia w danych, czy błąd parsera (na przykład cena 129900 zamiast 1299 przy złej obsłudze przecinka);
- wniosek — co ta analiza mówi o poprawności Twojego parsera i co poprawiłeś w jej wyniku.

Statystyki opisowe i wykresy

- średnia, mediana, moda, kwartyle, minimum, maksimum dla cech liczbowych;
- histogram co najmniej jednej cechy;
- wykres pudełkowy (box-plot) co najmniej jednej cechy;
- korelacja co najmniej dwóch cech liczbowych wraz z komentarzem — czy zależność ma sens merytoryczny.

Sprawozdanie

Sprawozdanie w formacie PDF ma zawierać:

- użytą technologię i uzasadnienie wyboru;
- opis zbudowanego serwisu (rola AUTOR) — struktura, sposób generowania, wybrane utrudnienia i powód wyboru;
- opis parsera (rola PARSER) — wybrane rozszerzenia oraz zastosowane selektory wraz z uzasadnieniem, dlaczego akurat takie;
- ocenę odporności własnych selektorów — które z nich przetrwałyby zmianę wyglądu serwisu, a które nie i dlaczego;
- wyniki kontroli jakości danych;
- omówienie statystyk i wnioski;
- dla poziomu 5.0 — porównanie surowego HTML z drzewem DOM.

Oddanie pracy

Na platformie Delta wgrajcie jedno archiwum na parę, o nazwie:

LAB1_Nazwisko_Imie_Nazwisko_Imie.zip

Poziomy 4.0 i 5.0 wymagają dodatkowo odpowiedzi ustnej na laboratoriach.

Wskazówki techniczne

- Python: requests + BeautifulSoup (parsowanie), pandas + matplotlib (analiza), Selenium (poziom 5.0).
- C# / .NET: HttpClient + HtmlAgilityPack, Selenium WebDriver na poziomie 5.0.
- Dowolny inny język i biblioteki są dopuszczalne - liczy się wynik i jakość selektorów.